

**PROYECTO FINAL
INTELIGENCIA ARTIFICIAL I**



Apellidos y Nombre: Vasquez Caiza Juan Marcelo.

Carrera: Ingeniería de Sistemas.

Materia: SIS 420.

Proyecto Final
Inteligencia Artificial I - SIS 420

1. Introducción y Objetivo

1.1. Problema: El proyecto busca predecir la cantidad de unidades vendidas de productos lácteos en un entorno minorista. El conjunto de datos original (dataset.csv) contiene 360,000 registros de transacciones diarias, detallando el producto, precio, descuento y fecha.

1.2. Objetivo: El objetivo principal es construir y entrenar un modelo de Regresión usando un Perceptrón Multicapa (MLP) con PyTorch. Este modelo debe ser capaz de predecir la cantidad de UnidadesVendidas para un producto específico en un día específico, basándose en las siguientes características:

- Precio: Variable importante en ventas, a mayor precio, menor demanda (ventas).
- Descuento: Mide la promoción o incentivo comercial, aumentar el descuento suele tener una relación directa con el aumento de ventas, ya que reduce el precio.
- Mes: Captura la estacionalidad y los eventos cíclicos. Las ventas varían entre los meses.
- Día de la semana: Captura el patrón semanal de compra. Es común que las ventas sean más altas los fines de semana.
- Producto(Nombre): Define qué artículo se está vendiendo (leche entera, yogurt, queso).
- Categoría(Tipo de Producto): Captura el patrón semanal de compra. Es común que las ventas sean más altas los fines de semana.

2. Metodología y Procesamiento de Datos

Para que el modelo MLP pudiera aprender patrones complejos, se siguió un riguroso proceso de preparación y estandarización de datos.

2.1. Carga y limpieza de datos

1. **Carga:** Se cargó el archivo dataset.csv en un DataFrame de Pandas.
2. **Limpieza:** Se verificó la existencia de datos nulos, confirmando que el conjunto de datos estaba completo.

2.2. Preparación de Features (X) y Target (y)

Este fue el paso más crítico para el éxito del modelo.

1. **Target (y):** La variable a predecir se definió como la columna UnidadesVendidas.

2. **Features (X):** Se definieron las características de entrada. Se eliminaron Fecha (redundante con Mes/Día) y UnidadesVendidas (el target).
3. **One-Hot Encoding:** Los MLPs no entienden texto. Por lo tanto, las columnas categóricas Producto y Categoría se convirtieron a un formato numérico usando `pd.get_dummies`. Esto expandió las features de 6 a 19 columnas, permitiendo al modelo diferenciar entre, por ejemplo, 'Leche Pil Entera 1L' y 'Yogurt Pil Fresa 1L'.

2.3. División y Estandarización

Para evitar la fuga de datos (data leakage) y asegurar un entrenamiento estable, se siguió el siguiente orden:

1. **División:** Los datos (X e y) se dividieron primero en conjuntos de entrenamiento (80%) y prueba (20%).
2. **Estandarización:** Se utilizó `StandardScaler` para transformar todas las features (X) a una media de 0 y desviación estándar de 1. El escalador se ajustó (`.fit()`) únicamente en `X_train` y luego se usó para transformar (`.transform()`) tanto `X_train` como `X_test`.
3. **Estandarización del Target:** De igual manera, la variable y (UnidadesVendidas) fue estandarizada usando su propio `scaler_y` (ajustado solo en `y_train`). Esto fue crucial para que el optimizador convergiera eficientemente.

3. Definición y entrenamiento del modelo

3.1. Arquitectura del MLP

Se definió una clase MLP personalizada que hereda, de [torch.nn.Module](#). La arquitectura es la siguiente:

1. **Capa de Entrada:** 19 neuronas (correspondientes a las 19 features de X).
2. **Capa Oculta 1:** 64 neuronas, seguidas de una función de activación ReLU.
3. **Capa Oculta 2:** 32 neuronas, seguidas de una función de activación ReLU.
4. **Capa de Salida:** 1 neurona (para predecir el valor único de UnidadesVendidas escalado).

3.2. Entrenamiento

1. **Función de Pérdida (Loss):** Se seleccionó `nn.MSELoss` (Error Cuadrático Medio), ya que es el estándar para problemas de regresión.
2. **Optimizador:** Se utilizó `optim.Adam` con un learning rate de 0.001. Adam es un optimizador robusto y adaptativo que funciona bien en la mayoría de los casos.

3. **Proceso:** El modelo y los tensores de datos se movieron a la GPU (.cuda()). El modelo se entrenó durante 4,500 épocas. El Loss (pérdida) convergió exitosamente, bajando de 0.92 a 0.09.

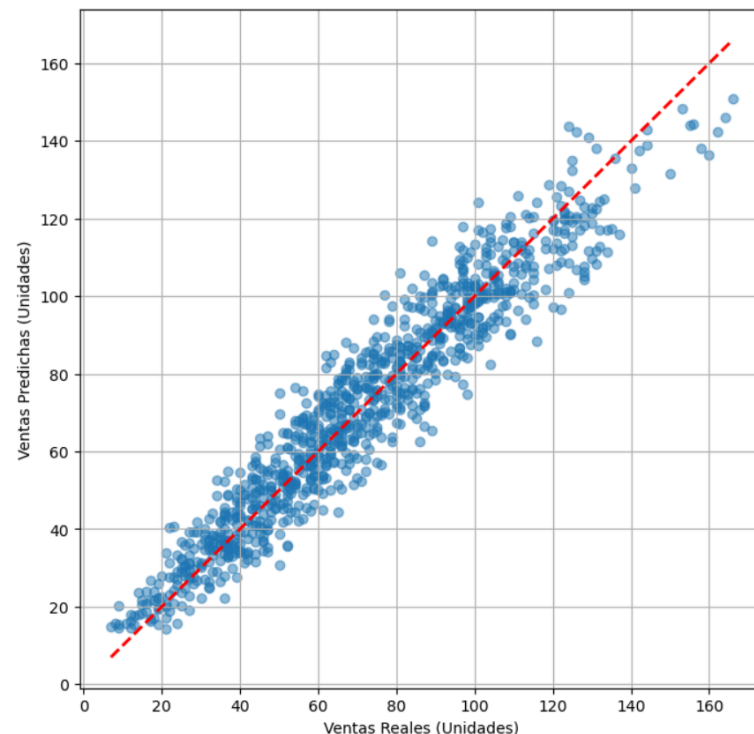
4. Evaluación y Resultados

Para evaluar el rendimiento del modelo en datos nunca vistos (el set de prueba), se usaron las siguientes métricas de regresión. Las predicciones escaladas se revirtieron a "Unidades Vendidas" usando `scaler_y.inverse_transform()`.

- **Coefficiente R-cuadrado (R^2): 0.9124**
 - **Interpretación:** Este es un resultado excelente. Significa que el modelo es capaz de explicar el 91.24% de la variabilidad en las ventas diarias.
- **Raíz del Error Cuadrático Medio (RMSE): 9.01 unidades**
 - **Interpretación:** Esta es la métrica de error más común. Indica que, en promedio, el error "típico" del modelo es de +/- 9 unidades.
- **Error Absoluto Medio (MAE): 7.07 unidades**
 - **Interpretación:** Esta es la métrica más intuitiva. Significa que, de media, las predicciones del modelo se desvían de la cantidad real en solo **7 unidades**.

Demostración Visual

La Tabla Comparativa (en el cuadernillo) y el Gráfico de Dispersión demuestran visualmente el éxito del modelo. El gráfico muestra cómo las predicciones (eje Y) se agrupan muy cerca de los valores reales (eje X), siguiendo la línea diagonal roja, lo que confirma el alto R^2 .



5. Sistema de Predicción (Cómo Usar el Modelo)

El notebook final incluye un sistema para realizar predicciones sobre datos nuevos. El proceso es el siguiente:

1. **Entrada:** El usuario define los datos de entrada (Precio, Descuento, Mes, Día, Producto, etc.) en un diccionario de Python.
2. **Pre-procesamiento:** El script aplica los mismos pasos de `get_dummies` y `reindex` que se usaron en el entrenamiento. Esto es crucial para asegurar que las columnas coincidan.
3. **Estandarización:** Los datos procesados se escalan usando el `scaler_X` guardado.
4. **Predicción:** El tensor escalado se pasa al modelo (`model.eval()`) para obtener una predicción escalada.
5. **Resultado:** La predicción escalada se revierte usando `scaler_y.inverse_transform()` para mostrar el resultado final en unidades vendidas.